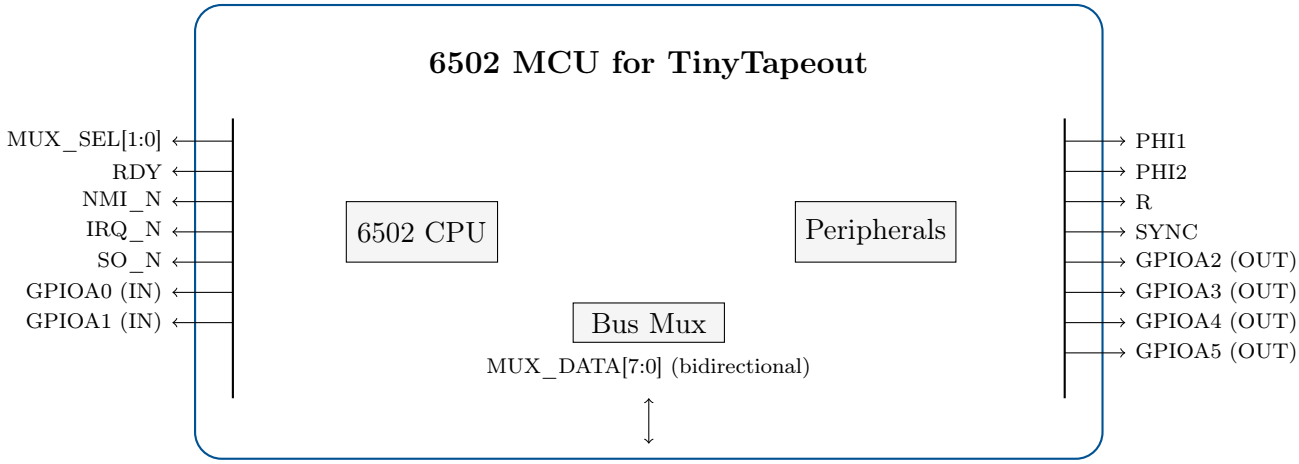
6502 Microcontroller

Complete CPU with Bus Multiplexer

GPIO, Timer, UART, and SK6812 LED Controller



Technical Datasheet

Document Version 1.0
February 14, 2026

Contents

1 Overview **2**
1.1 Key Features . . . 2
1.2 Target Applications . . . 2
1.3 Implementation Details . . . 2

2 System Architecture **3**
2.1 Block Diagram . . . 3
2.2 Memory Map . . . 3
2.3 Bus Multiplexer Protocol . . . 3

3 Pin Configuration **4**
3.1 Input Pins (ui_in[7:0]) . . . 4
3.2 Output Pins (uo_out[7:0]) . . . 4
3.3 Bidirectional Pins (uio[7:0]) . . . 4
3.4 GPIO Pin Mapping . . . 4

4 Peripheral Registers **5**
4.1 GPIO Peripheral (0xA000-0xA00B) . . . 5
4.1.1 Register Map . . . 5
4.1.2 Pin Mode Values . . . 5
4.1.3 Example: Configuring GPIO . . . 6
4.1.4 Example: Configuring UART on GPIO . . . 6
4.2 SK6812 LED Controller (0xA010-0xA017) . . . 6
4.2.1 Register Map . . . 6
4.2.2 Pin Configuration . . . 6
4.2.3 Protocol Sequence . . . 7
4.2.4 Example: Updating LED Strip . . . 7
4.3 Timer (0xA020-0xA027) . . . 8
4.3.1 Register Map . . . 8
4.3.2 Control Register (CTRL) . . . 8
4.3.3 Example: Periodic Timer with Polling . . . 8
4.4 Clock Control (0xA030-0xA033) . . . 9
4.4.1 Register Map . . . 9
4.4.2 Example: Power Saving . . . 10
4.5 UART (0xA040-0xA047) . . . 10
4.5.1 Register Map . . . 10
4.5.2 Control Register (CTRL) . . . 10
4.5.3 Status Register (STATUS) . . . 11
4.5.4 Baud Rate Formula . . . 11
4.5.5 Example: UART "Hello World" . . . 11

5 External Memory Interface **12**
5.1 Bus Multiplexer Operation . . . 12
5.2 Timing Diagram . . . 13
5.3 Required External Signals . . . 13
5.4 RP2040 PIO Example . . . 13

6 Getting Started **13**
6.1 Minimal System . . . 13
6.2 Reset Vector . . . 14
6.3 Example: LED Blink Program . . . 14
6.4 Toolchain . . . 15

7 Specifications **15**
7.1 Electrical Characteristics . . . 15
7.2 Timing Specifications . . . 15
7.3 Resource Utilization . . . 15

8 Revision History **16**

9 References **16**

1 Overview

The TinyTapeout 6502 Microcontroller is a complete MOS Technology 6502-compatible CPU with integrated peripherals, designed specifically for the TinyTapeout platform. This implementation features a bus multiplexer architecture that efficiently uses the limited 24-pin interface to expose the full CPU bus and peripheral functions.

1.1 Key Features

- **Complete 6502 CPU**: Cycle-accurate implementation with all documented opcodes
- **Bus Multiplexer**: 4-phase multiplexing reduces pin count from 24 to 8 data pins
- **External Memory Support**: Up to 64KB address space via multiplexed bus
- **Integrated Peripherals**:
  - GPIO: 6 pins (2 input-only, 4 output-only on TinyTapeout; internally bidirectional)
  - UART: 8N1 serial with 4-byte TX/RX FIFOs
  - Timer: 16-bit timer with prescaler and interrupt support
  - SK6812: Hardware RGB LED controller with pin multiplexing
  - Clock Control: Runtime CPU clock division
- **Pin Multiplexing**: UART, SK6812, and GPIO can share physical pins via mode registers
- **TinyTapeout Optimized**: Fits in 2×2 tile allocation with efficient gate usage

1.2 Target Applications

- Educational 6502 system on a chip
- Retro computing demonstrations
- LED display controllers with programmability
- Serial communication nodes
- Embedded control applications

1.3 Implementation Details

- **RTL Size**: 2,900 lines of SystemVerilog
- **Technology**: IHP SG13G2 130nm
- **Die Size**: 2×2 TinyTapeout tiles (419.52µm × 313.74µm)
- **Clock**: 50 MHz nominal (configurable)
- **CPU Speed**: Software-configurable from full speed down to 1/256× via clock divider

2 System Architecture

2.1 Block Diagram

The system consists of three major components:

1. **6502 CPU Core**: Executes instructions and drives the internal bus
2. **Bus Multiplexer**: Converts 16-bit address + 8-bit data to 8-bit multiplexed bus
3. **Memory-Mapped Peripherals**: GPIO, UART, Timer, SK6812 LED, Clock Control

The CPU accesses both internal peripherals and external memory through a unified 64KB address space. Address decoding logic routes accesses to 0xA000-0xA047 to internal peripherals, while all other addresses are multiplexed to the external bus.

2.2 Memory Map

Address Range	Size	Description
0x0000 – 0x9FFF	40KB	External memory (via bus mux)
0xA000 – 0xA00B	12 bytes	GPIO registers
0xA010 – 0xA017	8 bytes	SK6812 LED controller
0xA020 – 0xA027	8 bytes	Timer
0xA030 – 0xA033	4 bytes	Clock control
0xA040 – 0xA047	8 bytes	UART
0xA048 – 0xFFFF	22KB	External memory (via bus mux)

Table 1: System Memory Map

Typical usage allocates external RAM at 0x0000-0x7FFF and ROM at 0x8000-0xFFFF, with the 6502 reset vector at 0xFFFC pointing to boot code.

2.3 Bus Multiplexer Protocol

The bus multiplexer reduces pin requirements by time-multiplexing the address and data buses over 8 shared bidirectional pins. The external controller (e.g., RP2040 on TinyTapeout demo board) sequences through 4 phases within each CPU cycle:

MUX_SEL	Phase	Direction	Data on Bus
01	ADDR_HI	MCU → External	Address[15:8]
00	ADDR_LO	MCU → External	Address[7:0]
10	DATA_IN	External → MCU	Read data (if R/W=1)
11	DATA_OUT	MCU → External	Write data (if R/W=0)

Table 2: Bus Multiplexer Phases

All four phases complete within a single 6502 bus cycle, so there is **no performance penalty** compared to a parallel bus. The external controller monitors PHI2 to track cycle boundaries and controls MUX_SEL accordingly.

3 Pin Configuration

The TinyTapeout interface provides 24 pins total: 8 dedicated inputs (ui_in), 8 dedicated outputs (uo_out), and 8 bidirectional pins (uio). This implementation uses all available pins efficiently:

3.1 Input Pins (ui_in[7:0])

Pin	Function
ui_in[0]	MUX_SEL[0] – Bus phase select bit 0
ui_in[1]	MUX_SEL[1] – Bus phase select bit 1
ui_in[2]	RDY – CPU ready signal (active high)
ui_in[3]	NMI_N – Non-maskable interrupt (active low)
ui_in[4]	IRQ_N – Interrupt request (active low)
ui_in[5]	SO_N – Set overflow flag (active low)
ui_in[6]	GPIOA0 – GPIO pin 0 input (unidirectional)
ui_in[7]	GPIOA1 – GPIO pin 1 input (unidirectional)

Table 3: Input Pin Assignments

3.2 Output Pins (uo_out[7:0])

Pin	Function
uo_out[0]	PHI1 – CPU phase 1 clock
uo_out[1]	PHI2 – CPU phase 2 clock
uo_out[2]	R/W – Read/write signal (1=read, 0=write)
uo_out[3]	SYNC – Opcode fetch indicator
uo_out[4]	GPIOA2 – GPIO pin 2 output (unidirectional)
uo_out[5]	GPIOA3 – GPIO pin 3 output (unidirectional)
uo_out[6]	GPIOA4 – GPIO pin 4 output (unidirectional)
uo_out[7]	GPIOA5 – GPIO pin 5 output (unidirectional)

Table 4: Output Pin Assignments

3.3 Bidirectional Pins (uio[7:0])

Pin	Function
uio[7:0]	MUX_DATA[7:0] – Multiplexed address/data bus

Table 5: Bidirectional Pin Assignments

The MUX_DATA pins are driven by the MCU during ADDR_HI, ADDR_LO, and DATA_OUT phases, and tri-stated during DATA_IN phase to allow the external controller to drive read data.

3.4 GPIO Pin Mapping

While the MCU's internal GPIO peripheral is fully bidirectional, the TinyTapeout pin assignment splits GPIO across input and output pins:

- **GPIO 0-1**: Input-only (read via INPUT register at 0xA002)
- **GPIO 2-5**: Output-only (write via OUTPUT register at 0xA001, controlled by OE register at 0xA000)

Each GPIO pin has an independent mode register (0xA004-0xA009) that can configure it as:

- GPIO mode (0x00) – Standard I/O
- UART TX mode (0x01) – UART transmit output
- UART RX mode (0x02) – UART receive input
- SK6812 DATA mode (0x03) – LED controller output

See Section 4 for complete GPIO peripheral documentation.

4 Peripheral Registers

4.1 GPIO Peripheral (0xA000-0xA00B)

4.1.1 Register Map

Address	Register	Description
0xA000	OE	Output Enable (0=input, 1=output)
0xA001	OUT	Output Data
0xA002	IN	Input Data (read-only)
0xA003	Reserved	–
0xA004	MODE_PIN0	Pin 0 function select
0xA005	MODE_PIN1	Pin 1 function select
0xA006	MODE_PIN2	Pin 2 function select
0xA007	MODE_PIN3	Pin 3 function select
0xA008	MODE_PIN4	Pin 4 function select
0xA009	MODE_PIN5	Pin 5 function select
0xA00A	MODE_PIN6	Reserved (unused)
0xA00B	MODE_PIN7	Reserved (unused)

Table 6: GPIO Register Map

Note: Only GPIO 0-5 are connected to TinyTapeout pins. GPIO 6-7 registers exist but are not connected.

4.1.2 Pin Mode Values

Value	Function
0x00	GPIO – Standard I/O controlled by OE/OUT
0x01	UART0_TX – UART transmit output
0x02	UART0_RX – UART receive input
0x03	SK6812_DATA – LED controller output
0x04-0xFF	Reserved

Table 7: GPIO Pin Mode Values

4.1.3 Example: Configuring GPIO

```
; Set GPIO2 as output, GPIO3 as output
LDA #$0C          ; Bits 2 and 3
STA $A000         ; Write to OE register

; Set GPIO2 high, GPIO3 low
LDA #$04          ; Bit 2
STA $A001         ; Write to OUT register

; Read GPIO0 and GPIO1
LDA $A002         ; Read IN register
AND #$03          ; Mask bits 0-1
```

Listing 1: Configure GPIO pins

4.1.4 Example: Configuring UART on GPIO

```
; Set GPIO2 as UART TX
LDA #$01          ; UART0_TX mode
STA $A006         ; MODE_PIN2

; Set GPIO3 as UART RX
LDA #$02          ; UART0_RX mode
STA $A007         ; MODE_PIN3
```

Listing 2: Configure UART TX on GPIO2 and RX on GPIO3

4.2 SK6812 LED Controller (0xA010-0xA017)

Hardware controller for SK6812/WS2812 addressable RGB LEDs with precise timing.

4.2.1 Register Map

Address	Register	Description
0xA010	CONTROL	Control register (write 1 to start)
0xA011	CLKDIV	Clock divider for bit timing
0xA012	RED	Red component (0-255)
0xA013	GREEN	Green component (0-255)
0xA014	BLUE	Blue component (0-255)
0xA015	WHITE	White component (0-255, unused for RGB)
0xA016	STATUS	Status register (bit 0 = BUSY)
0xA017	Reserved	–

Table 8: SK6812 Register Map

4.2.2 Pin Configuration

The SK6812 data output must be routed to a GPIO pin using the mode register:

```
LDA #$03          ; SK6812_DATA mode
STA $A008         ; MODE_PIN4
```

Listing 3: Configure SK6812 output on GPIO4

4.2.3 Protocol Sequence

To update an LED:

1. Wait for BUSY=0 (read STATUS register, bit 0)
2. Write RED, GREEN, BLUE values (WHITE is ignored for RGB LEDs)
3. Write 0x01 to CONTROL to start transmission
4. Repeat for each LED in chain
5. After last LED, wait 50
   mus+ for latch

4.2.4 Example: Updating LED Strip

```
; Configure GPIO4 as LED data output
LDA #$03
STA $A008          ; MODE_PIN4 = SK6812_DATA

; LED 0: Red
wait_0:
    LDA $A016      ; Read STATUS
    AND #$01       ; Check BUSY
    BNE wait_0     ; Wait if busy

LDA #$FF
STA $A012          ; RED = 255
LDA #$00
STA $A013          ; GREEN = 0
STA $A014          ; BLUE = 0
LDA #$01
STA $A010          ; Start transmission

; LED 1: Green
wait_1:
    LDA $A016
    AND #$01
    BNE wait_1

LDA #$00
STA $A012          ; RED = 0
LDA #$FF
STA $A013          ; GREEN = 255
LDA #$00
STA $A014          ; BLUE = 0
LDA #$01
STA $A010

; LED 2: Blue
wait_2:
    LDA $A016
    AND #$01
    BNE wait_2

LDA #$00
STA $A012          ; RED = 0
```

```
STA $A013          ; GREEN = 0
LDA #$FF
STA $A014          ; BLUE = 255
LDA #$01
STA $A010

; Wait for latch (50$\\mu$s+ idle time)
; Implement delay loop here
```

Listing 4: Update 3-LED RGB strip

4.3 Timer (0xA020-0xA027)

16-bit up-counter with prescaler, auto-reload, and interrupt support.

4.3.1 Register Map

Address	Register	Description
0xA020	CTRL	Control register
0xA021	STATUS	Status register (bit 0 = OVERFLOW)
0xA022	COUNT_LO	Counter low byte (read-only)
0xA023	COUNT_HI	Counter high byte (read-only)
0xA024	RELOAD_LO	Reload value low byte
0xA025	RELOAD_HI	Reload value high byte
0xA026	PRESCALER	Clock prescaler (0-255)
0xA027	Reserved	–

Table 9: Timer Register Map

4.3.2 Control Register (CTRL)

- Bit 0: ENABLE – Enable timer counting
- Bit 1: AUTO_RELOAD – Reload from RELOAD_LO/HI on overflow
- Bit 2: IRQ_ENABLE – Enable interrupt on overflow
- Bit 3: LOAD – Load counter from reload registers (write-only strobe, only works when ENABLE=0)
- Bits 7-4: Reserved

4.3.3 Example: Periodic Timer with Polling

```
; Calculate prescaler for 1ms tick at 50MHz
; Desired: 1000 ticks per second
; Timer freq = 50MHz / (PRESCALER + 1)
; For 50kHz: PRESCALER = 999
; Overflow period = 65536 / 50000 = 1.31s

; Better: PRESCALER = 49 for 1MHz tick rate
; Then set RELOAD to get 1ms period (1000 ticks)

init_timer:
```

```
    ; Set prescaler for 1MHz tick
    LDA #49             ; sysclk / 50 = 1MHz
    STA $A026           ; PRESCALER

    ; Set reload for 1000 ticks = 1ms
    ; Reload value = 65536 - 1000 = 64536 = 0xFC18
    LDA #$18
    STA $A024           ; RELOAD_LO
    LDA #$FC
    STA $A025           ; RELOAD_HI

    ; Load counter and enable with auto-reload
    LDA #$08            ; LOAD (timer stopped)
    STA $A020
    LDA #$03            ; ENABLE | AUTO_RELOAD
    STA $A020

timer_loop:
    ; Poll for overflow
    LDA $A021           ; Read STATUS
    AND #$01            ; Check OVERFLOW
    BEQ timer_loop      ; Wait for overflow

    ; Clear overflow flag
    LDA #$01
    STA $A021

    ; Do 1ms periodic work here
    JSR do_work

    JMP timer_loop

do_work:
    ; Your code here
    RTS
```

Listing 5: 1ms periodic timer at 50MHz

4.4 Clock Control (0xA030-0xA033)

Runtime CPU clock division for dynamic frequency scaling.

4.4.1 Register Map

Address	Register	Description
0xA030	CPU_DIV	CPU clock divisor (0-255)
0xA031	Reserved	–
0xA032	STATUS	Clock status (bit 0 = CPU_LOCKED)
0xA033	Reserved	–

Table 10: Clock Control Register Map

CPU Clock Formula: cpu_freq = cpu_clk / (CPU_DIV + 1)

Where cpu_clk is the hardware-divided clock (configured by CPU_CLOCK_HW_DIV parameter).

4.4.2 Example: Power Saving

```
; Reduce CPU speed to 1/16 while idle
idle_loop:
    LDA #$0F          ; Divide by 16
    STA $A030         ; CPU_DIV

    ; Poll for event at slower speed
wait_event:
    LDA $A002         ; Read GPIO
    AND #$01          ; Check button
    BEQ wait_event

    ; Restore full speed
    LDA #$00
    STA $A030         ; CPU_DIV = 0 (full speed)

    ; Handle event
    JSR handle_event
    JMP idle_loop
```

Listing 6: Slow down CPU while waiting

4.5 UART (0xA040-0xA047)

8N1 serial communication with 4-byte TX/RX FIFOs and configurable baud rate.

4.5.1 Register Map

Address	Register	Description
0xA040	CTRL	Control register
0xA041	STATUS	Status register
0xA042	DATA	Data register (FIFO access)
0xA043	BAUD_LO	Baud divisor low byte
0xA044	BAUD_HI	Baud divisor high byte
0xA045-0xA047	Reserved	–

Table 11: UART Register Map

4.5.2 Control Register (CTRL)

- Bit 0: TX_ENABLE – Enable transmitter
- Bit 1: RX_ENABLE – Enable receiver
- Bit 2: TX_IRQ_EN – Enable TX ready interrupt
- Bit 3: RX_IRQ_EN – Enable RX ready interrupt
- Bits 7-4: Reserved

4.5.3 Status Register (STATUS)

- Bit 0: TX_READY – TX FIFO not full (can write)
- Bit 1: RX_READY – RX FIFO not empty (can read)
- Bit 2: TX_EMPTY – TX FIFO completely empty
- Bit 3: RX_FULL – RX FIFO full
- Bit 4: TX_ACTIVE – Currently transmitting
- Bit 5: RX_ERROR – Framing error detected
- Bits 7-6: Reserved

4.5.4 Baud Rate Formula

baud_rate = sysclk / (16 × (divisor + 1))

Common baud rates at 50MHz:

Baud Rate	Divisor	BAUD_HI:LO
9600	325	0x01:0x45
19200	162	0x00:0xA2
38400	80	0x00:0x50
57600	53	0x00:0x35
115200	26	0x00:0x1A

Table 12: Common Baud Rates (50MHz sysclk)

4.5.5 Example: UART "Hello World"

```
; Configure GPIO2 as UART TX
LDA #$01          ; UART0_TX mode
STA $A006         ; MODE_PIN2

; Initialize UART for 9600 baud @ 50MHz
init_uart:
    ; Set baud rate (divisor = 325 = 0x0145)
    LDA #$45
    STA $A043         ; BAUD_LO
    LDA #$01
    STA $A044         ; BAUD_HI

    ; Enable TX
    LDA #$01          ; TX_ENABLE
    STA $A040         ; CTRL

; Transmit "Hello\r\n"
send_hello:
    LDX #0
tx_loop:
    LDA message, X
    BEQ tx_done

    ; Wait for TX ready
tx_wait:
```

```
    LDA $A041          ; Read STATUS
    AND #$01           ; Check TX_READY
    BEQ tx_wait        ; Wait if FIFO full

    LDA message, X
    STA $A042          ; Write byte
    INX
    BNE tx_loop

tx_done:
    RTS

message:
    .byte "Hello", $0D, $0A, 0  ; CR+LF+null
```

Listing 7: Transmit string via UART

5 External Memory Interface

5.1 Bus Multiplexer Operation

The external controller (typically an RP2040 on the TinyTapeout demo board) must:

1. Monitor PHI2 to detect cycle boundaries
2. Wait for address setup time (tADS
   approx 150ns after falling PHI2)
3. Sequence MUX_SEL through address phases:
   - MUX_SEL = 01 → Latch ADDR[15:8] from MUX_DATA
   - MUX_SEL = 00 → Latch ADDR[7:0] from MUX_DATA
4. During PHI2 high, provide or capture data:
   - If R/W = 1 (read): Set MUX_SEL = 10, drive MUX_DATA with memory contents
   - If R/W = 0 (write): Set MUX_SEL = 11, capture MUX_DATA to memory

All phases complete within one CPU cycle. At 1MHz CPU clock, each phase has approximately 175ns for address and 500ns for data.

5.2 Timing Diagram

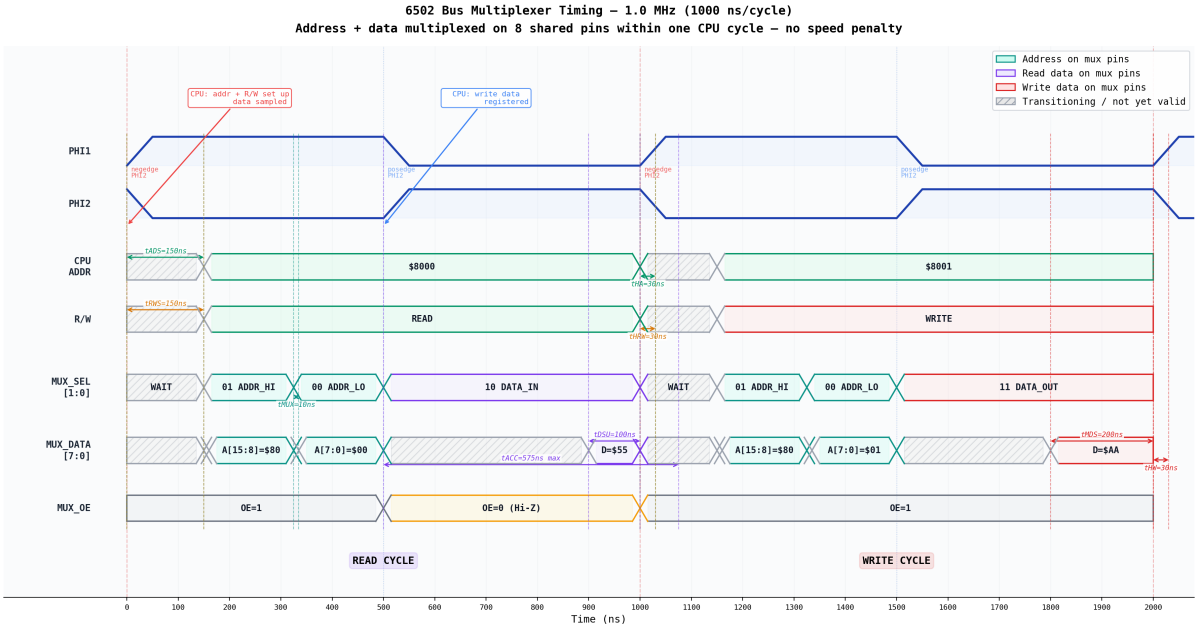


Figure 1: Bus Multiplexer Timing Diagram (1MHz CPU clock)

5.3 Required External Signals

The external memory controller needs these signals from the MCU:

- **PHI2** (uo_out[1]): CPU clock for cycle tracking
- **R/W** (uo_out[2]): Read (1) or write (0)
- **SYNC** (uo_out[3]): Optional, indicates opcode fetch
- **MUX_DATA** (uio[7:0]): Bidirectional multiplexed bus

And drives:

- **MUX_SEL[1:0]** (ui_in[1:0]): Phase select
- **MUX_DATA** (uio[7:0]): Drive during read data phase

5.4 RP2040 PIO Example

The TinyTapeout demo board includes an RP2040 that can serve as the memory controller using its Programmable I/O (PIO) subsystem. A complete reference implementation is available in the upstream m6502 project.

6 Getting Started

6.1 Minimal System

A minimal working system requires:

1. **External Memory Controller**: RP2040 or similar

2. **Power & Clock**: 3.3V supply, 50MHz clock
3. **Reset**: Active-low reset signal
4. **Memory Contents**: ROM image with 6502 code and reset vector

6.2 Reset Vector

On reset, the 6502 CPU:

1. Loads the program counter from addresses 0xFFFC-0xFFFD (reset vector)
2. Begins executing code at that address

Ensure your ROM image contains a valid reset vector pointing to your startup code.

6.3 Example: LED Blink Program

```
.org $8000              ; ROM starts at 0x8000

reset:
    ; Configure GPIO2 as output
    LDA #$04            ; Bit 2
    STA $A000           ; OE register

main_loop:
    ; Toggle GPIO2
    LDA $A001           ; Read current output
    EOR #$04            ; Toggle bit 2
    STA $A001           ; Write back

    ; Delay
    JSR delay
    JMP main_loop

delay:
    LDY #$00
delay_outer:
    LDX #$00
delay_inner:
    NOP
    DEX
    BNE delay_inner
    DEY
    BNE delay_outer
    RTS

; Reset vector
.org $FFFC
.word reset             ; Reset vector points to reset routine
.word reset             ; NMI vector (reuse reset for simplicity)
```

Listing 8: Simple LED blink on GPIO2

6.4 Toolchain

Common 6502 assemblers and tools:

- **cc65**: C compiler and assembler suite
- **ACME**: Cross-assembler
- **xa65**: Cross-assembler
- **py65**: Python-based 6502 simulator for testing

7 Specifications

7.1 Electrical Characteristics

Parameter	Min	Typ	Max
Supply Voltage	1.1V	1.2V	1.3V
I/O Voltage	3.0V	3.3V	3.6V
Clock Frequency	DC	50MHz	100MHz
Power Consumption	–	TBD	TBD

Table 13: Electrical Characteristics

7.2 Timing Specifications

At 50MHz system clock:

Parameter	Value	Notes
CPU Clock (default)	50MHz	Configurable via CPU_CLOCK_HW_DIV
CPU Clock (min)	195kHz	CPU_DIV = 255
UART Baud Rate	9600-115200	Standard rates supported
Timer Resolution	20ns	At 50MHz sysclk, PRESCALER=0
SK6812 Bit Period	Configurable	Set via CLKDIV register

Table 14: Timing Specifications

7.3 Resource Utilization

Estimated gate count breakdown (2×2 TinyTapeout tiles):

Component	Approximate Lines
6502 CPU Core	923
Bus Multiplexer	44
GPIO Peripheral	135
UART (TX+RX+Main)	472
Timer	151
SK6812 LED	188
Clock Control	72
MCU Integration	216
Total RTL	**2,900 lines**

Table 15: RTL Size Breakdown

8 Revision History

Version	Date	Changes
1.0	February 14, 2026	Initial release for TinyTapeout

9 References

- MOS Technology 6502 Programming Manual
- Western Design Center W65C02S Datasheet
- TinyTapeout Documentation: `https://tinytapeout.com`
- Upstream m6502 Project: `https://github.com/chrismoos/6502-mcu`